Parse, Retrieve, Cite — then design your own agent

ME/CE/AM 295

Agentic AI in Scientific Discovery
9 units (3-5-1)

Instructor: Chiara Daraio
TAs: Xiaoxiao (Alice) Xiong
Hyoseo (Lauren) Yoon

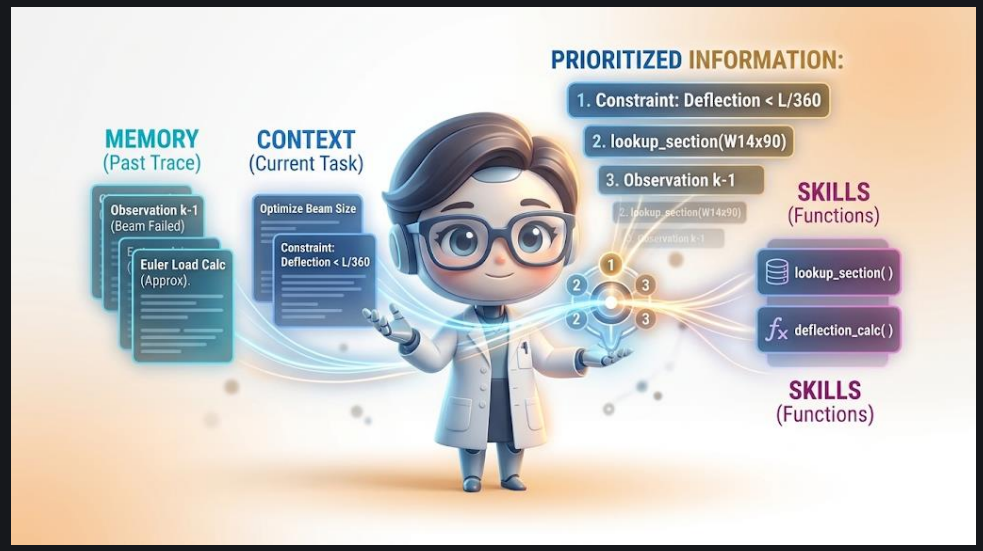
California Institute of Technology
Division of Engineering and Applied Science

Lecture 8

Monday 26 October 2026

WHERE WE LEFT OFF · WEDNESDAY (LECTURE 7)

The agent runs at 07:07, fetches the day's arXiv, remembers in files you merge. It can fetch a feed. It cannot answer a question from a document.



PART 1
Why the agent must read

PART 2
How retrieval works

PART 3
Design your own agent

1 why the agent must read documents → 2 where the corpus lives, how retrieval works, and Lab 5 → break → 3 what Lab Report 1 told us, four design rules, your own agent on paper, a pair review

The Deliberate Failure — Ask It Before It Can Look Up

One question to the capstone agent. It has fetched a thousand abstracts; it has nothing to look this up in. Hold the answer; we come back to it on the cite-and-check slide.

THE QUESTION

"What deflection limit applies to a floor supporting a plaster ceiling, under live load?"

```
# terminal, inside capstone-agent/  (not at Claude's prompt)
claude -p "<the question>" --permission-mode dontAsk

# -> "The typical limit is L/360 for live load on floors
#     with plaster ceilings; some codes use L/240 for
#     total load ..."

#   No code named.  No table.  No edition.  No footnote.
#   beam_tools.py's limit_in = 0.667 is a constant in the
#   script — not a lookup either.
```

WATCH

A fluent answer with no provenance. Plausible is the failure mode — a wrong answer that looks like a right one.

WHICH ROW

The semantic row of the decision table: facts the agent did not write. They must be retrieved, cited, checked — never remembered.

HOLD IT

The cite-and-check slide asks this exact question again, with a retriever and a schema that forces a citation — or an abstain.

Nothing in that answer was retrieved. It may be right. You cannot tell — and neither can the agent.

Memory Is Not Retrieval

Your agent's choice of a 20 ft span is memory. ASCE 7's drift limit is retrieval. Both can live in the same vector database; they differ in who wrote them and how they fail.

MEMORY — what the agent did

Who wrote it the agent, and you

Trust a decision — may be wrong, but it is the record

Failure stale: recalls a load case that has changed

Fix re-insert current constraints from state

`USER.md · MEMORY.md`

`checkpoint · thread_id`

RETRIEVAL — what others wrote

Who wrote it authors, standards bodies, your lab

Trust a source — may be misread, never edited

Failure wrong passage: § 12.8 when asked about § 12.9

Fix cite the section; check it against the source

`corpus/ (PDFs, in git)`

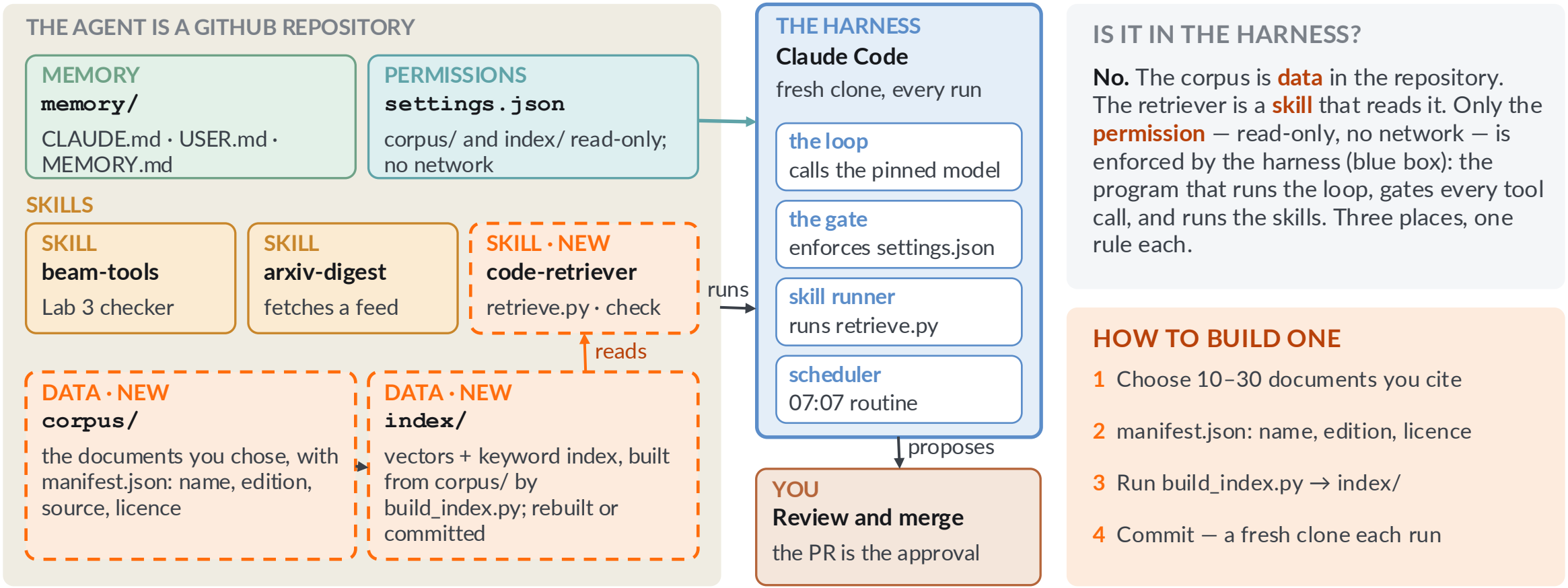
`index/ (vector + BM25)`

"We gave it long-term memory with RAG" means it can read documents. It still forgets what it decided yesterday.

Chapter 3 § 3.19 and Chapter 4 § 4.4. Design rule for today: the retriever has no write path to memory/; memory never holds a code value.

Where the Corpus Lives — Inside the Agent You Built

A corpus is the folder of documents the agent is allowed to look things up in. It is data in the repository — not memory, not the harness. A skill reads it; the harness only says who may touch it.



IS IT IN THE HARNESS?

No. The corpus is **data** in the repository. The retriever is a **skill** that reads it. Only the **permission** — read-only, no network — is enforced by the harness (blue box): the program that runs the loop, gates every tool call, and runs the skills. Three places, one rule each.

HOW TO BUILD ONE

1. Choose 10–30 documents you cite
2. manifest.json: name, edition, licence
3. Run build_index.py → index/
4. Commit — a fresh clone each run

Blue box = the harness: the program that runs the agent. It reads files from the repository (settings.json, skills) and enforces them; it never reads corpus/ itself — the retriever skill does. Orange = new: corpus/, index/, code-retriever.

The Pipeline, and Where Each Stage Breaks

Seven stages. Each fails silently — the run completes, the number looks plausible, and it is wrong. The next three slides take the first six in pairs; Lab 5 builds all seven.

1	2	3	4	5	6	7
Parse	**Chunk**	**Embed**	**Index**	**Retrieve**	**Cite**	**Check**
pdf → text + structure	text → units + metadata	chunk → vector	vectors + keyword index, in git	query → top-k	chunks → typed answer	quote ∈ chunk ?
BREAKS AS shredded table	BREAKS AS lost footnote	BREAKS AS A992 ≈ A36	BREAKS AS not in the clone	BREAKS AS wrong passage	BREAKS AS invented §	BREAKS AS — the fix

Four of the six failures are retrieval failures. The generator is usually innocent. When an answer is wrong, walk this row before you touch the prompt.

Stages 1–4 build index/ from corpus/ (once, committed); stages 5–7 run inside the code-retriever skill on every question. **When an answer is wrong, walk this row before you touch the prompt.**

Today's example corpus, and Lab 5's: AISC Table 1-1 (W-shapes), ASCE 7 Ch. 12, IBC Table 1604.3, three arXiv PDFs — in corpus/ and index/ of the capstone-agent repository; a fresh clone each cloud run, so anything not committed does not exist.

Ingestion Decides Retrieval Quality — Parse, then Chunk

A PDF stores glyph positions, not structure. What the parser keeps and the chunker attaches is all a citation can ever contain. Lost here, unrecoverable downstream.

Shape	Area, A	Depth, d	Web		Flange		Distance				
			Thickness, t_w	$\frac{t_w}{2}$	Width, b_f	Thickness, t_f	k_{des}	k_{det}	k_1	T	Workable Gage
	in.²	in.	in.	in.	in.	in.	in.	in.	in.	in.	in.
W10×112	32.9	11.4	11 3/8	0.755	3/4	3/8	10.4	10 3/8	1.25	1 1/4	1.75	1 15/16	1	7 1/2	5 1/2
×100	29.3	11.1	11 1/8	0.680	11/16	3/8	10.3	10 3/8	1.12	1 1/8	1.62	1 13/16	1		
×88	26.0	10.8	10 7/8	0.605	5/8	5/16	10.3	10 1/4	0.990	1	1.49	1 11/16	15/16		
×77	22.7	10.6	10 5/8	0.530	1/2	1/4	10.2	10 1/4	0.870	7/8	1.37	1 9/16	7/8		
×68	19.9	10.4	10 3/8	0.470	1/2	1/4	10.1	10 1/8	0.770	3/4	1.27	1 7/16	7/8		
×60	17.7	10.2	10 1/4	0.420	7/16	1/4	10.1	10 1/8	0.680	11/16	1.18	1 3/8	13/16		
×54	15.8	10.1	10 1/8	0.370	3/8	3/16	10.0	10	0.615	5/8	1.12	1 5/16	13/16		
×49	14.4	10.0	10	0.340	5/16	3/16	10.0	10	0.560	9/16	1.06	1 1/4	13/16		
W10×45	13.3	10.1	10 1/8	0.350	3/8	3/16	8.02	8	0.620	5/8	1.12	1 5/16	13/16	7 1/2	5 1/2
×39	11.5	9.92	9 7/8	0.315	5/16	3/16	7.99	8	0.530	1/2	1.03	1 3/16	13/16		
×33	9.71	9.73	9 3/4	0.290	5/16	3/16	7.96	8	0.435	7/16	0.935	1 1/8	3/4		

the page as the parser sees it — AISC Table 1-1, Dimensions, W10 rows

Ask: "web thickness of W10×49?" t_w = 0.340 in. Only the right-hand parse can find it — in the stream, "W10x49" never occurs.

```
# text-stream parser
"W10×112 32.9 11.4 11 3/8 ...
 ... ×49 14.4 10.0 10 0.340
 5/16 3/16 10.0 10 0.560 ..."
```

shredded — the row for W10×49 does not contain "W10"

```
# layout-aware parser
{table: "1-1 Dimensions",
 row: {Shape: "W10x49",
       A: 14.4, d: 10.0,
       tw: 0.340}}
```

structure kept — the group header "W10" reaches every row

CHUNK — the same clause, cut two ways

By character count



every N characters: the table is cut from the footnote that governs it

By structure



at § and table boundaries: heading, table and footnote stay together

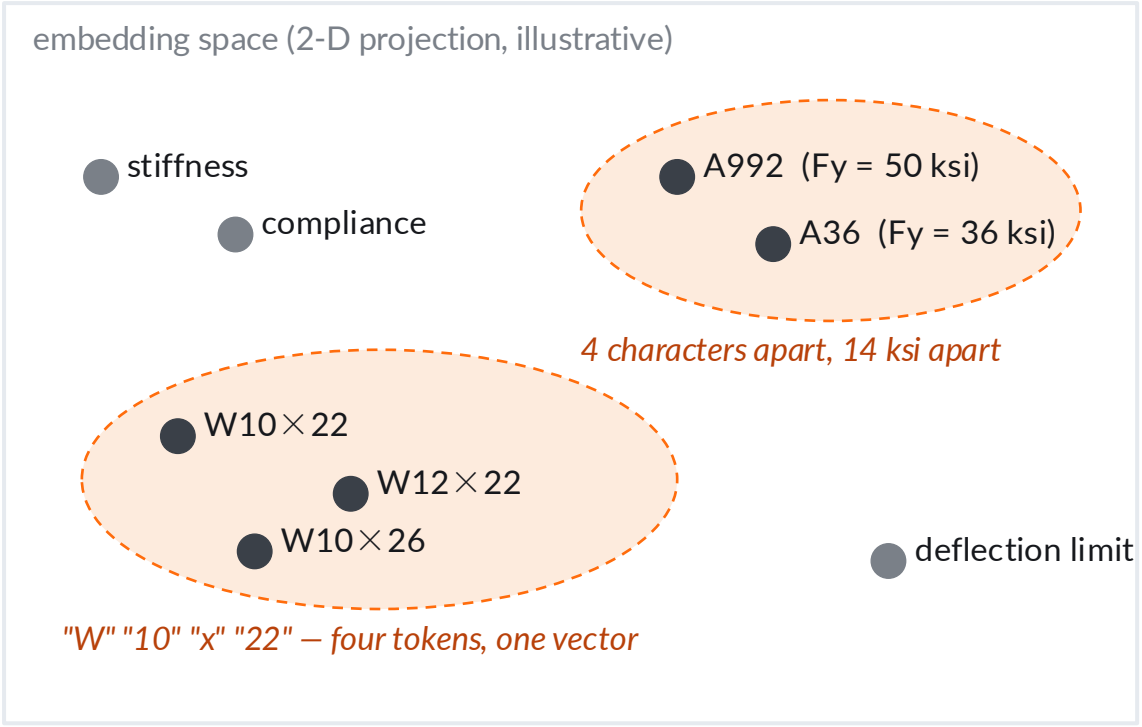
orange = table + footnote · other chunkers: Ch. 5

Metadata on every chunk — doc · edition · section · page · table — is the citation. Structure in, citation out.

Parsers: pdfplumber / PyMuPDF (fast; table grids) · Unstructured / Docling (layout-aware; keeps hierarchy) · Nougat (equations). Chunk overlap 10–20 %. Cross-references ("see § 12.8.7") die at chunk boundaries; parent-document expansion recovers them at retrieval time.

Search Two Ways, or It Misses the Identifier

An embedding places a chunk near chunks used in similar sentences — a proxy for meaning, and a leaky one. Engineering text is full of exact tokens that embeddings blur: W10×22, A992, § 12.8.7.



Question asked	Embedding search	Keyword search	Both, fused
I_x of W10x22 (exact identifier)	W12×22 ✗	W10×22 ✓	W10×22 ✓
allowable floor sag (paraphrase)	1604.3 ✓	no hit ✗	1604.3 ✓

TWO WAYS TO SEARCH THE INDEX

Row 1: embedding search blurs the identifier — W10×22 and W12×22 are nearly one vector; keyword search matches the exact string.

Row 2: keyword search has no hit — "sag" is not in the clause; embedding search knows sag ≈ deflection.

Opposite failures — so run both and merge the ranked lists.

Run both searches and merge the ranks ("hybrid"). Engineering text is exact tokens; one search alone is not enough.

Ranks from the pre-class dry run on the course corpus; re-measure before class. Fusion, re-ranking and MMR: Chapter 5 and the Lab 5 handout. Embeddings: self-hosted Sentence-Transformers (no key) or Voyage AI (network secret).

Cite Under a Schema, Check with a Script

Answer from context, or decline. The schema forces a citation or an abstain; a script — not a model — verifies that every quote is in the chunk it cites and the edition matches the pin.

```
class Grounded(BaseModel):
    answer: str
    citations: list[Citation]   # >= 1
    abstain: bool = False
    reason: str | None = None
```

`Citation` = doc · edition · section · page · verbatim quote (≤ 25 words). No citation, no well-formed answer — rejected before anyone reads it.

SLIDE 3'S QUESTION, ASKED AGAIN — pre-run transcript

L/360 for live load on floor members supporting plaster ceilings; L/240 for total load.

`citations[0]` IBC 2024 · Table 1604.3 · p. 16-7 · "Floor members … L/360 … L/240"

Asked about Eurocode 3, which is not in the corpus: **`abstain = true`** — the honest verdict a confident number would have hidden.



Schema-valid is still not grounded: a model can invent a citation that parses (Ch. 2 § 2.16, now for citations). Run the pipeline twenty times and count PASS / FAIL / ABSTAIN — the tally goes into Lab Report 2. A FAIL is not a bug report; it is the evidence the Checker exists to produce.

RAG, Long Context, or Agentic Search?

Three ways to put a document in front of the model. Which one depends on what you have and what you ask.

WHAT YOU HAVE	LONG CONTEXT	RAG · today	AGENTIC SEARCH
	one document, one question	**a library, many questions**	**"has anyone tried X?"**
e.g.	*one OpenSees manual page, sent once*	*your lab's 300 papers plus the codes*	*"this alloy above 600 ° C — anyone?"*
WHAT IT DOES	paste it, question last	retrieve top-k chunks, then answer	model queries, reads, queries again
Who picks the context	you	**the retriever**	the model
The citation is	a position in the paste	**chunk metadata: doc · page**	the tool trace
Cost per question	highest: whole document, each time	**bounded by k**	open-ended; can loop
Fails as	lost in the middle	**wrong passage**	wrong query, endless loop
Suits a corpus that is	small and static	**large and changing**	live and mixed

Worksheet box 5: most of you will write "retriever over my corpus" as a tool — correct. The agentic version is a loop around that tool, not a different tool.

Capstone rule: start with RAG; promote to agentic search only when one top-k cannot answer; long context only for a single document sent once. Agentic search = retrieval inside the loop — an orchestration pattern (Chapter 5).

Lab 5 — Give Your Agent Something to Read

Everything the last three slides described, built on your own project's documents and registered as skill #3 of your capstone agent. Handout on Slack tonight; it feeds Lab Report 2.

WHAT A CORPUS IS

The folder of documents your agent is allowed to look things up in — 10 to 30 PDFs your project actually cites: the standard you design to, the papers your method rests on, your lab's own reports. You choose them; you commit them, with a manifest (name, edition, source, licence). Not the web, and not what the model was trained on.

THE ONE NUMBER

25 questions from your documents, checked by hand. Context precision: of the chunks returned, how many were the right ones. One search vs both merged. 0.6 → 0.9 is a result; "it seems better" is not.

A	**Build your corpus** 10–30 documents + manifest.json; decide what may be committed	45 min
B	**Parse, chunk, index** run the build; open one chunk you know and check its structure survived	30 min
C	**Register it as skill #3** SKILL.md · settings.json · PERMISSIONS.md § 5 · one call in dontAsk whose transcript shows both scripts	30 min
D	**The question set and the number** 25 hand-checked questions; precision for one search vs both; the 20-run citation tally	90 min
E	**What the retriever is for** the semantic row of your decision table; tool, research question, or not needed	15 min

No document corpus in your project? Do Parts A–E on the course corpus (AISC Table 1-1, ASCE 7 Ch. 12, IBC 1604.3, three arXiv PDFs — in the template) and say so. That is a complete submission.

5-minute break

WHEN WE COME BACK

Lab 1 Reports Debrief and Discussion

Lab Report 1 — What 27 Reports Told Us

The full debrief is on Slack. Half the class has a pipeline, not an agent — here is the evidence, and the rest of today is the fix, applied to each project individually. Lab Report 2 (the proposal) is due Sunday 1 November.

WHAT THE CLASS DID WELL

- ✓ **Deterministic checks outside the model** a checker decides pass/fail; the agent plans and interprets
- ✓ **Non-agent baselines** scripted or optimizer baselines; several ablation ladders
- ✓ **Credential and model hygiene** git-ignored .env, secret scans, pinned model IDs, approval points
- ✓ **Serious domain tools** real simulators and hardware, with real failure modes

½ keep the LLM out of the verifier

½ named a non-agent baseline

5 used an agent for literature search — and the two who verified quote, page and DOI caught real errors

0 **built retrieval over their own paper corpus — Lab 5 is the recipe**

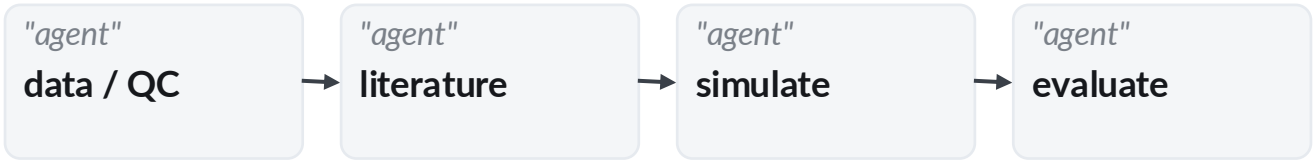
The cover-up test: cover the word "agent" on each box of your diagram. Could twenty lines of Python produce the same output from the same input? Then it is a tool. Diagnosing, choosing, proposing, abstaining — that is where the agent belongs.

From the Lab Report 1 debrief (Week 4). Domain tools in the room: OpenSeesPy, OpenDrift, ngspice, COMSOL, LSDEM, MuJoCo, Stokesian dynamics, tactile hardware — real tools with real failure modes.

Lab Report 1 — Drawn Twice, and Before Sunday

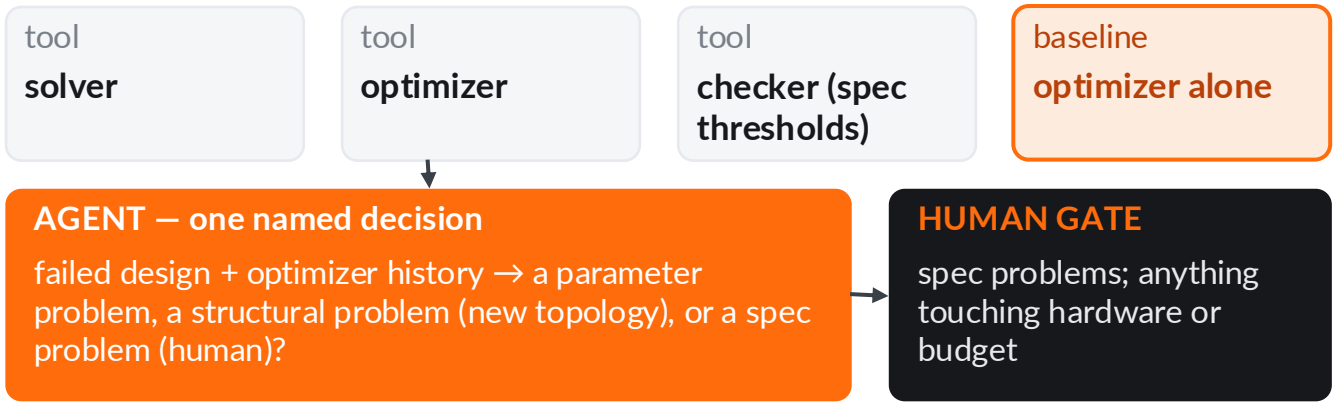
A made-up project: pick the cell depth and perforation ratio of a honeycomb acoustic liner so a duct attenuates a target band, with a transfer-matrix solver.

VERSION A — most Lab Report 1 diagrams



four boxes, zero agent decisions once you cover the word

VERSION B — after the cover-up test



DESIGN RULES FOR LAB REPORT 2

1 Tool first, smoke-tested and typed; then the scripted baseline; only then the agent.

2 Write down roles, tool interfaces, state and memory, hand-offs, stopping conditions, approval points.

3 Define evaluation before building: held-out tasks, repeated runs, pinned model, abstention counted.

4 Derive every threshold from the spec or the math — never by eye.

5 Validate every tool call against a schema; explicit human approval for hardware, cluster jobs, budgets.

6 Scope for nine weeks, not a thesis. Small and verified beats broad and unverifiable.

Blockers we are fixing: Python on AWS WorkSpaces; MATLAB/COMSOL/Abaqus licences; local MCP servers blocked by policy; API keys. Raise yours with the TAs on Slack this week, not in Lab Report 2.

Rule 1 — Start From What You Cannot Script

Do not start from the four-box template. Start from the workflow you already run by hand, and mark where you stop and think.

THE PROCEDURE

1. **Write the workflow as you actually do it:** raw input to the number you report, one step per line.
2. **Mark each step R or D.** R = run it: a fixed input gives a fixed output (load a file, run the solver, compute RMSE). D = look and decide: you read an output and choose what happens next.
3. **Every R step becomes a tool.** Every D step is an agent candidate.
4. **Pick the one D step** whose outcome you can check and whose baseline you can run. That is your agent. Everything else is a tool it calls.

D STEPS FROM YOUR LAB REPORT 1s

- Naming the fault behind a failed check and choosing a remedy, or declining to report
- Deciding whether a counterexample calls for a gain change or a new controller structure
- Choosing where in a five-dimensional space the next simulation campaign should go
- Judging which of 300 abstracts deserve a full read, and when to stop searching
- Deciding that the spec itself is wrong and asking a person

If your project is truly a pipeline, consider building an agent that operates, monitors and audits that pipeline, or individual steps within it

Worksheet boxes 1 and 2. The beam agent of Lecture 6 passes this test: lookup and deflection are R; "which section next, given this failure" is D.

Rule 2 — Cut at Verification Boundaries, Not Task Boundaries

Not the retrieval chunks of Part 2 — the boxes in your diagram. "How big should each piece be?" is the wrong question. A piece is the right size when something that is not the LLM can check its output.

1

The piece has a check that is not the LLM

A formula (deflection, equilibrium, residual vs noise floor), a unit test, a held-out measurement, a human label. Keep it. The check's threshold comes from the spec or the math, never from the model.

2

The piece has no check

Split it until each part has one — or it is not an agent decision at all: make it a tool. Pieces that cannot be checked individually are where errors compound silently, and nothing raises.

3

The only possible check is another LLM

Allowed where no formula exists ("is this summary accurate?"), but you have bought a second faithfulness problem. Give the critic different information from the worker, and put a deterministic or external check at the end.

4

The test, in one sentence

Point at each box and finish: "I know this box was right because ___." If the blank is "the model said so", the box is not done.

Worksheet box 3. Lecture 6's Checker is this rule at the scale of one loop; here it decides the whole architecture. The Lab 1 smoke test that failed because a tolerance was guessed: the integrator was right and the check was wrong.

Rule 3 — One Agent per Context, Not per Task

The office-hours question was "how specialized should each agent be". Split into a second agent only for one of three reasons; otherwise it is a pipeline stage wearing a hat.

SPLIT · INFORMATION

The two jobs would mislead each other if one agent held both. The Checker must not see the Designer's reasoning, or it rationalizes it. Demo B's Checker got files, not the Researcher's transcript.

SPLIT · PERMISSIONS

The thing that launches cluster jobs or moves a robot arm should not be the thing that edits the spec. Different allow-lists in settings.json means different agents.

SPLIT · JUDGMENT

You need a second opinion on the same object: an adversary hunting for a counterexample, a reviewer with no memory of the project. Same model, different context: independence of context, not of judgment.

DO NOT SPLIT FOR

Pipeline stages (load, clean, run, tabulate): those are tools. **"It feels cleaner":** that is a cost with no evidence. **Generalist vs specialist:** a generalist with many tools is fine for exploring; a specialist with few tools and a typed contract is what you can evaluate, reproduce and audit in Week 9. Build the specialist.

Worksheet box 6. Most capstone projects need one or two LLM agents and three to five tools. Four agents and zero tools is the ratio backwards.

Rule 4 — An Agent Is a Folder in Git, Not a Conversation

The office-hours question was "how do I set them up and redeploy them". You already built the answer in Lab 4; most of you have not connected it to your project.

PART OF THE AGENT	WHERE IT LIVES IN YOUR REPO	WHAT GOES THERE
Spec and the "may not" list	`CLAUDE.md, PERMISSIONS.md`	What the agent is for, what it may not do; read at every start
Tools	`.claude/skills/<name>/SKILL.md + scripts`	One job each, units in the argument names, inputs echoed in every result
Permissions	`.claude/settings.json`	Allow / ask / deny, enforced by the harness, not the model
Memory	`memory/USER.md, MEMORY.md, daily/`	Dated, sourced, merged by you; never holds a retrieved value
Run record	`thread_id in the checkpoint; LOG.md`	One thread per component revision; the log your agents read and append
Environment	`requirements.lock, pinned model ID`	Requirements pinned in one lock file; the model named in config, not in code

Worksheet box 7. The redeploy test: can a fresh clone on another machine run your smoke test with one command? If the agent exists only as a Claude Code session you keep open, it is not set up yet.

The Four Rules on a Messy Project — a Filled-in Worksheet

An imaginary project, chosen because it has none of the beam's conveniences. The same seven boxes you fill

THE PROJECT A biologist records rattlesnake rattles in the field and wants to classify each recording by the temperature it was made at (snakes rattle faster when warm). 600 clips. The temperature labels came from the site logbooks and some are wrong. There is no formula that says whether a label is right.

R = run it: a fixed input gives a fixed output → a tool. **D** = look and decide: you read an output and choose what happens next → an agent candidate. (Rule 1)

1 Workflow by hand

1–3 load, segment, features — R
4 doubtful label: keep / relabel / discard — D
5 train the classifier — R
6 read the confusion matrix — D
7 report accuracy + CI — R

2 The one decision

Step 4. A clip whose label disagrees with its rattle rate: keep, relabel (with a reason), or discard. Stop when every flagged clip is resolved. Escalate to the PI if a whole site looks wrong.

3 Its check — not the LLM

60 clips labelled independently by two people (κ > 0.8): relabel precision and recall against them. Plus a physical bound: the rattle rate must fit the published range for that temperature. Both are scripts.

4 Its baseline — no agent

A fixed rule on the same features: noisy clip → discard; rate outside the bound → relabel; else keep. Same held-out clips, five repeats. The agent must beat it — or the rule suffices, and that is the result.

5 Tool list (in → out, units)

segment_rattles(wav) → intervals [s]
features(interval) → rate [Hz], SNR [dB]
rate_bound(T [° C]) → [Hz] range
train_eval(split) → accuracy, 95 % CI
All exist? Y — steps 1–3, 5, 7.

6 A second agent? Why?

No. Step 6 is also a D step, but it needs the same information, the same permissions and the same judgment as step 4 — none of Rule 3's three reasons. It stays with the human for now.

7 The redeploy test

CLAUDE.md — spec, may-not list
skills/ — four tools · settings.json — no network
memory/ by PR · LOG.md · lock file · model pinned
make smoke — from a fresh clone

2 D STEPS OF 7.
1 AGENT, 4 TOOLS.
A CHECK THAT IS NOT THE MODEL.
A BASELINE TO BEAT.

The Worksheet — on Paper

One page, your own project. Short phrases. Where a box does not apply, write why. Then swap with a neighbour: finish "I know this box was right because ___" for their boxes 2 and 3.

1 Your workflow by hand

one step per line;
mark each R or D

2 The one decision

input, options a/b/c, stop or escalate

3 Its check

not the LLM; threshold from spec or math

4 Its baseline

same tools, no agent

5 Tool list

name, in → out with units, exists? Y/N

6 A second agent? Why?

different information, permissions, or judgment

7 The redeploy test

which file holds spec, tools, permissions, memory; fresh clone, one command

8 One question for a TA

read first tonight

WHAT HAPPENS NEXT

On your own, then the pair review, then hand in at the door. TAs read every sheet tonight; Wed: three redrawn live, then the clinic.

Boxes 2, 3 and 4 are the core of Sunday's one-page proposal. Box 7 is what the TAs check for participation. Box 8 is read first tonight. Photograph your sheet before you hand it in.

From the Worksheet to the Proposal

What the teaching team does with your sheet, and what you do before Sunday 1 November.

MON NIGHT

TAs read every worksheet

Sorted into four piles: decision + check + baseline named · decision without a check · no decision yet · the decision is literature search and the literature is support, not the research loop.

WED · LECTURE 9

Design clinic

Four rules recap, three composite redraws, then three of your worksheets redrawn live. Orchestration patterns taught on your diagrams, not on a framework survey.

WED · OFFICE HOUR

TA clinic

Right after the lecture. Bring the redrawn diagram. You leave with a yes or a no on whether it meets the proposal bar, and one concrete next step.

SUN 1 NOV

Lab Report 2

One page: research question, the agent's decision, the domain tool, baseline, evaluation, risks, team. Map the worksheet onto the repo: D step → agent spec, R steps → skills, check → script, baseline → second entry point. Repo public, log active.

Blockers we are fixing: Python on AWS WorkSpaces; MATLAB/COMSOL/Abaqus licences; local MCP servers blocked by policy; API keys. Raise yours on Slack this week, not in Lab Report 2.

Lecture 8 — Takeaways, Lab, and Wednesday

The case

Attention, cost, provenance. Retrieval is the mechanism that makes "do not use an unretrieved number" enforceable.

The build

Structure is not in the file · one search alone misses the identifier · a citation is a claim a script checks. Lab 5 builds the whole pipeline on your documents; fixes are settings and scripts, never prompts.

The thesis, and the workshop

Grounding is a property of the pipeline; a citation ≠ evidence until checked. For your own project: one named decision, a check that is not the LLM, a baseline with the same tools, an agent that lives in a folder in git.

LAB THIS WEEK · LAB 5

Add a retriever over your own project's documents to your capstone agent, and show one number that says it works.
Your corpus + manifest · skill #3 registered · 25 hand-checked questions · context precision, one search vs both · the 20-run citation tally.
Handout: Lab 5, Parts A–E → Lab Report 2.

WEDNESDAY · LECTURE 9

Design clinic: four rules recap

The three orchestration patterns you need, taught on your diagrams.

A real life example.

References — Lecture 8

Required: Skarlinski et al. (2024), Language agents achieve superhuman synthesis of scientific knowledge (PaperQA2), arXiv:2409.13740 · Brown et al. (2025) [Ref_06] · Liu et al. (2024), Lost in the Middle, TACL 12, § 4–5.

Supplementary: Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP, NeurIPS · Es et al. (2023), RAGAS, arXiv:2309.15217 · Gao et al. (2024), RAG for LLMs: a survey, § 3–4 · Karpukhin et al. (2020), Dense Passage Retrieval · Robertson & Zaragoza (2009), BM25 · Anthropic (2024), Contextual Retrieval (engineering post) · Willison (2025), The lethal trifecta.

Tool documentation (fetch date to add): LangChain loaders, text splitters, vector stores · Chroma · FAISS · Docling · Unstructured · Nougat · Sentence-Transformers · Voyage AI · Claude Code skills and permissions.

Before Wednesday: Schluntz & Zhang (2024), Building Effective Agents [Ref_20] · Cemri et al. (2025), MAST § § 1–4 [Ref_25] · Elrefaie et al. (2025) [Ref_05].

FIGURES TO EXTRACT FROM THE REFERENCES

Slide 7 · A screenshot of the AISC Table 1-1 page beside its parsed output (course-made; check AISC terms for classroom use).

Chapter 5 · Anthropic (2024) Contextual Retrieval post — the preamble-prepending diagram.

Slide 8 · Lewis et al. (2020) Fig. 1 — the original RAG architecture (retriever + generator).

Chapter 5 · Skarlinski et al. (2024) Fig. 1 — the PaperQA2 agentic search loop (CC BY).

Chapter 5 · Liu et al. (2024) Fig. 1 (lost in the middle) and Willison (2025) (the trifecta) — no longer on slides; use in the study guide.

Cross-references: Lecture 4 (tokenization, Table 2.2, lost in the middle) · Lecture 5 (cost model) · Lecture 6 (decision table, Designer/Checker) · Lecture 7 (fresh-clone rule, skills, permissions, trifecta).